

**Practicum Partition., Proof of Set Propositions.,
Multisets., Set Type in Pascal Language., Introduction to
Logic and Fuzzy Sets**

WEEK 7

Subject:

Computational Mathematics

Prepared by:

Aisyah Currents

25091397108

Class International/Number 2



INFORMATICS MANAGEMENT STUDY PROGRAM

FACULTY OF VOCATION

UNIVERSITAS NEGERI SURABAYA

YEAR 2025

Table of Contents

PRACTICUM Part 1: Applications of Equivalent Sets., Disjoint Sets., Power Sets., Operations on Sets.....	3
PRACTICUM Part 2: Applications of Equivalent Sets., Disjoint Sets., Power Sets., Operations on Sets in Python.	7
PRACTICUM Part 3: Case studies in the Theory of Equivalent Sets., Disjoint Sets., Power Sets., Operations on Sets.	11
PRACTICUM Part 4: Practice Exercises of Equivalent Sets., Disjoint Sets., Power Sets., Operations on Sets.....	15
PRACTICUM Part 5: Applications Extension of Set Operations., Laws of Sets., Principle of Duality., Principle of Inclusion-Exclusion.	20
PRACTICUM Part 6: Case studies in the Theory of Extension of Set Operations., Laws of Sets., Principle of Duality., Principle of Inclusion-Exclusion.	23
PRACTICUM Part 7: Practice Exercises of Extension of Set Operations., Laws of Sets., Principle of Duality., Principle of Inclusion-Exclusion.	27
PRACTICUM Part 8: Applications Partition., Proof of Set Propositions., Multisets., Set Type., Introduction to Logic and Fuzzy Sets.	32
PRACTICUM Part 9: Case Study in the Theory of Partition., Proof of Set Propositions., Multisets., Set Type., Introduction to Logic and Fuzzy Sets.	35
PRACTICUM Part 10: Practice Exercises of Partition., Proof of Set Propositions., Multisets., Set Type., Introduction to Logic and Fuzzy Sets.	39

PRACTICUM Part 1: Applications of Equivalent Sets., Disjoint Sets., Power Sets., Operations on Sets.

1. Equivalent Sets

Equivalent sets are two sets that have the same number of elements, even though the elements themselves are different. In other words, if the size (cardinality) of two sets is the same, then the sets are called equivalent.

The Source Code with output

```
Praktikum 1 > 1.py > ...
1  set_a = {1, 2, 3}
2  set_b = {3, 4, 5}
3  print(len(set_a)==len(set_b))
4  print("Size set A:", len(set_a))
```



```
True
Size set A: 3
```

The Analysis

In the given Python source code, two sets are defined: `set_a = {1, 2, 3}` and `set_b = {3, 4, 5}`. The program then compares the lengths of both sets using the expression `len(set_a) == len(set_b)` to determine whether the two sets are equivalent in terms of their number of elements. Since both sets contain three elements, the comparison evaluates to `True`, which is printed as the first line of output. The next line prints the size of `set_a`, which is 3. This demonstrates the concept of equivalent sets, where two sets are considered equivalent if they have the same cardinality, regardless of the specific elements contained within them.

2. Disjoint Sets

Two sets are said to be disjoint if there are no common elements between them. In Python, the `.isdisjoint()` function is used to check whether two sets are disjoint.

The Source Code with output

```
Praktikum 1.py > ...
5
6  set_a = {1, 2, 3}
7  set_b = {3, 4, 5}
8  print(set_a.isdisjoint(set_b))
```

```
Documents/Kuliah
False
PS C:\Users\USER
tkom>
```

The Analysis

In the given Python program, two sets are defined as `set_a = {1, 2, 3}` and `set_b = {3, 4, 5}`. The function `isdisjoint()` is used to check whether the two sets have no elements in common. If the sets share any element, the function returns `False`; otherwise, it returns `True`. In this case, both sets contain the element 3, meaning they are not disjoint. Therefore, the output of the program is `False`, indicating that there is at least one common element between `set_a` and `set_b`. This demonstrates that `isdisjoint()` helps identify whether two sets are completely separate or share overlapping elements.

3. Power Set

A power set is the set of all possible subsets of a given set. In Python, we can use functions from the `itertools` module to generate a power set.

The Source Code with output

```
Praktikum 1.py > ...
10 from itertools import chain, combinations
11
12 def power_set(s):
13     return list(chain.from_iterable(combinations(s, r) for r in range(len(s)+1)))
14
15 set_a = {1, 2, 3}
16 print(power_set(set_a))

[(), (1,), (2,), (3,), (1, 2), (1, 3), (2, 3), (1, 2, 3)]
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika
```

The Analysis

In the given Python program, the function `power_set(s)` is defined to generate all possible subsets of a given set using the `itertools` module. Specifically, the `combinations()` function is used within a loop to create subsets of every possible

length, starting from 0 up to the total number of elements in the set. The `chain.from_iterable()` function combines all these subsets into a single iterable list. When the function is applied to `set_a = {1, 2, 3}`, it produces the power set of that set, which includes all possible combinations: the empty set `()`, single-element subsets `(1)`, `(2)`, `(3)`, two-element subsets `(1, 2)`, `(1, 3)`, `(2, 3)`, and the complete set `(1, 2, 3)`. The output confirms that the function correctly lists every subset, demonstrating how Python can efficiently generate a power set using built-in combinatorial tools.

4. Operations on Sets

The basic operations on sets include:

- **Union:** Combines all elements from two sets.
- **Intersection:** Contains only the elements that exist in both sets.
- **Difference:** Contains elements that exist in one set but not in the other.
- **Complement:** Contains elements that exist in the universal set but not in a specific set.

Union ($\cup$)

Combines the elements from both sets (`set_a  $\cup$  set_b`).

Intersection ($\cap$)

Contains the elements that are present in both sets (`set_a  $\cap$  set_b`).

Difference ($-$)

Contains the elements that exist in one set but not in the other (`set_a - set_b`).

Complement

Contains the elements that are in the universal set but not in the specific set. For `complement_a`, these are the elements not in `set_a`, and for `complement_b`, these are the elements not in `set_b`.

[The Source Code with output](#)

```

Praktikum 1.py > ...
18 # Himpunan A dan B
19 set_a = {1, 2, 3, 4}
20 set_b = {3, 4, 5, 6}
21
22 # Himpunan semesta adalah gabungan dari set_a dan set_b
23 himpunan_semesta = set_a.union(set_b)
24
25 # Operasi Gabungan (Union)
26 union = set_a.union(set_b)
27
28 # Operasi Irisan (Intersection)
29 intersection = set_a.intersection(set_b)
30
31 # Operasi Selisih (Difference)
32 difference_a_b = set_a.difference(set_b)
33
34 # Operasi Komplemen (Complement)
35 complement_a = himpunan_semesta.difference(set_a)
36 complement_b = himpunan_semesta.difference(set_b)
37
38 # Menampilkan hasil operasi
39 print(f"Set A: {set_a}")
40 print(f"Set B: {set_b}")
41 print(f"Gabungan (A ∪ B): {union}")
42 print(f"Irisan (A ∩ B): {intersection}")
43 print(f"Selisih (A - B): {difference_a_b}")
44 print(f"Komplemen (Semesta - A): {complement_a}")
45 print(f"Komplemen (Semesta - B): {complement_b}")
46

```

```

Set A: {1, 2, 3, 4}
Set B: {3, 4, 5, 6}
Gabungan (A ∪ B): {1, 2, 3, 4, 5, 6}
Irisan (A ∩ B): {3, 4}
Selisih (A - B): {1, 2}
Komplemen (Semesta - A): {5, 6}
Komplemen (Semesta - B): {1, 2}

```

The Analysis

In this Python program, various set operations are demonstrated using two sets, `set_a = {1, 2, 3, 4}` and `set_b = {3, 4, 5, 6}`. The program first defines the universal set (`himpunan_semesta`) as the union of both sets, containing all unique elements from `set_a` and `set_b`. The union operation (`set_a.union(set_b)`) combines all elements from both sets without duplicates, resulting in `{1, 2, 3, 4, 5, 6}`. The intersection operation (`set_a.intersection(set_b)`) finds elements common to both sets, which are `{3, 4}`. The difference operation (`set_a.difference(set_b)`) shows elements in `set_a` that are not in `set_b`, producing `{1, 2}`. The complement operation is calculated relative to the universal set, showing which elements of the universal

set are not in each individual set. Thus, the complement of A (`himpunan_semesta.difference(set_a)`) is {5, 6}, and the complement of B (`himpunan_semesta.difference(set_b)`) is {1, 2}. The output correctly displays all these results, confirming that Python's set methods effectively perform standard set operations such as union, intersection, difference, and complement.

PRACTICUM Part 2: Applications of Equivalent Sets., Disjoint Sets., Power Sets., Operations on Sets in Python.

1. Equivalent Sets

Two sets are said to be equivalent if they have the same number of elements, regardless of whether the elements themselves are the same or not.

The Source Code with output

```
Praktikum 2.py > ...
1 def equivalent_sets():
2     print("=== Himpunan yang ekuivalen ===")
3     set_a = {1, 2, 3, 4}
4     set_b = {5, 6, 7, 8}
5
6     # Cek apakah kedua himpunan ekuivalen (memiliki ukuran yang sama)
7     are_equivalent = len(set_a) == len(set_b)
8     print(f"Set A: {set_a}")
9     print(f"Set B: {set_b}")
10    print(f"Apakah kedua himpunan ekuivalen? {are_equivalent}")
11    print("=====\n")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERM
=== Himpunan yang ekuivalen ===
Set A: {1, 2, 3, 4}
Set B: {8, 5, 6, 7}
Apakah kedua himpunan ekuivalen? True
=====
```

The Analysis

The program defines a function called `equivalent_sets()` to determine whether two sets have the same number of elements. It creates two sets, `set_a = {1, 2, 3, 4}` and `set_b = {8, 5, 6, 7}`, then compares their lengths using `len(set_a) == len(set_b)`. Although the elements differ, both sets contain four members, so the comparison returns `True`, indicating they are equivalent in size. This output shows that the program correctly demonstrates the concept of set equivalence, where two sets are

considered equivalent if they have the same number of elements, even if their contents are not identical.

2. Disjoint Sets

Two sets are said to be disjoint if there are no common elements between them.

The Source Code with output

```
Praktikum 2.py > ...
13 def disjoint_sets():
14     print("=== Himpunan yang saling lepas ===")
15     set_a = {1, 2, 3}
16     set_b = {4, 5, 6}
17
18     # Cek apakah kedua himpunan saling lepas (tidak memiliki elemen yang sama)
19     are_disjoint = set_a.isdisjoint(set_b)
20     print(f"Set A: {set_a}")
21     print(f"Set B: {set_b}")
22     print(f"Apakah kedua himpunan saling lepas? {are_disjoint}")
23     print("=====\n")
```

```
=== Himpunan yang saling lepas ===
Set A: {1, 2, 3}
Set B: {4, 5, 6}
Apakah kedua himpunan saling lepas? True
=====
```

The Analysis

The program defines a function called `disjoint_sets()` to check whether two sets have no common elements. It creates two sets, `set_a = {1, 2, 3}` and `set_b = {4, 5, 6}`, then uses the method `set_a.isdisjoint(set_b)` to determine if they are disjoint. Since none of the elements in `set_a` appear in `set_b`, the result is `True`, meaning the two sets are completely separate. This program effectively demonstrates the concept of disjoint sets in set theory, where two sets are considered disjoint if they share no common members.

3. Power Set

The power set of a set is the set that consists of all possible subsets of that set, including the empty set and the set itself.

The Source Code with output


```

Praktikum 2.py > ...
25 def power_set(s):
26     print("=== Himpunan Kuasa ===")
27     from itertools import chain, combinations
28
29     def power_set_generator(s):
30         return list(chain.from_iterable(combinations(s, r) for r in range(len(s)+1)))
31
32     # Himpunan kuasa dari himpunan s
33     p_set = power_set_generator(s)
34     print(f"Himpunan: {s}")
35     print(f"Himpunan Kuasa: {p_set}")
36     print("=====\n")
37

```

```

=== Himpunan Kuasa ===
Himpunan: {1, 2, 3}
Himpunan Kuasa: [(), (1,), (2,), (3,), (1, 2), (1, 3), (2, 3), (1, 2, 3)]
=====

```

The Analysis

The program defines a function called `power_set(s)` that generates the power set of a given set `s`, which is the collection of all possible subsets of that set. It uses Python's `itertools` library, specifically `chain` and `combinations`, to create combinations of every possible length from 0 up to the size of the set. For example, if the input set is `{1, 2, 3}`, the resulting power set includes all subsets: `()`, `(1,)`, `(2,)`, `(3,)`, `(1, 2)`, `(1, 3)`, `(2, 3)`, and `(1, 2, 3)`. The output confirms that the program correctly lists every possible combination, demonstrating the concept of a power set, which in set theory represents the total set of all subsets of a given set, including the empty set and the set itself.

4. Operations on Sets

The basic operations on sets include union, intersection, difference, and complement.

The Source Code with output

```

Praktikum 2.py > ...
38 def set_operations():
39     print("=== Operasi Himpunan ===")
40     # Himpunan A dan B
41     set_a = {1, 2, 3, 4}
42     set_b = {3, 4, 5, 6}
43
44     # Himpunan semesta adalah gabungan dari set_a dan set_b
45     himpunan_semesta = set_a.union(set_b)
46
47     # Operasi Gabungan (Union)
48     union_set = set_a.union(set_b)
49
50     # Operasi Irisan (Intersection)
51     intersection_set = set_a.intersection(set_b)
52
53     # Operasi Selisih (Difference)
54     difference_set = set_a.difference(set_b)
55
56     # Operasi Komplemen (Complement)
57     complement_a = himpunan_semesta.difference(set_a)
58     complement_b = himpunan_semesta.difference(set_b)
59
60     # Menampilkan hasil operasi
61     print(f"Set A: {set_a}")
62     print(f"Set B: {set_b}")
63     print(f"Gabungan (A ∪ B): {union_set}")
64     print(f"Irisan (A ∩ B): {intersection_set}")
65     print(f"Selisih (A - B): {difference_set}")
66     print(f"Komplemen A (Semesta - A): {complement_a}")
67     print(f"Komplemen B (Semesta - B): {complement_b}")
68     print("=====\n")
69

```

```

=== Operasi Himpunan ===
Set A: {1, 2, 3, 4}
Set B: {3, 4, 5, 6}
Gabungan (A ∪ B): {1, 2, 3, 4, 5, 6}
Irisan (A ∩ B): {3, 4}
Selisih (A - B): {1, 2}
Komplemen A (Semesta - A): {5, 6}
Komplemen B (Semesta - B): {1, 2}
=====

```

The Analysis

The program defines a function called `set_operations()` that performs several basic set operations using two sets, `set_a = {1, 2, 3, 4}` and `set_b = {3, 4, 5, 6}`. It first creates a universal set by combining both sets, then calculates the union, intersection, difference, and complement. The union ($A \cup B$) combines all unique elements from both sets, while the intersection ($A \cap B$) finds elements common to both sets. The difference ($A - B$) gives elements in A that are not in B, and the complements are found by subtracting each set from the universal set. The output correctly shows these results, demonstrating how Python's set methods can

represent fundamental set theory concepts such as union, intersection, difference, and complement in a clear and practical way.

5. Main execution

The Source Code

```
69
70 if __name__ == "__main__":
71     equivalent_sets()
72     disjoint_sets()
73     power_set({1, 2, 3})
74     set_operations()
```

The Analysis

The main execution section serves as the driver part of the program, which calls all the previously defined functions in sequence. When the file is executed, the `if __name__ == "__main__":` statement ensures that the program runs only when it is executed directly, not when imported as a module. Inside this block, the functions `equivalent_sets()`, `disjoint_sets()`, `power_set({1, 2, 3})`, and `set_operations()` are executed one after another. As a result, the program displays outputs for each set operation, including checking equivalence, disjointness, power sets, and set operations such as union, intersection, difference, and complement.

PRACTICUM Part 3: Case studies in the Theory of Equivalent Sets., Disjoint Sets., Power Sets., Operations on Sets.

1. Identifying Equivalent Sets

A lecturer wants to compare two lists of course participants for two different classes, namely “Course A” and “Course B.” The lecturer wants to know whether both classes have the same number of participants, regardless of their names. Therefore, the lecturer can check whether the two sets are equivalent by comparing the size (cardinality) of the two sets.

The Source Code with output

```

Praktikum 3.py > ...
1  kursus_a = {"Deny", "Budi", "Citra", "Dewi"}
2  kursus_b = {"Fahri", "Gina", "Hadi", "Ira"}
3
4  # Cek apakah kedua himpunan ekuivalen
5  def cek_himpunan_ekivalen(set_a, set_b):
6      return len(set_a) == len(set_b)
7
8  if cek_himpunan_ekivalen(kursus_a, kursus_b):
9      print("Kedua himpunan ekuivalen.")
10 else:
11     print("Kedua himpunan tidak ekuivalen.")

```

Kedua himpunan ekuivalen.

The Analysis

The program compares two sets of course participants, `kursus_a` and `kursus_b`, to determine whether they are equivalent. Each set contains the names of students enrolled in different classes, and the function `cek_himpunan_ekivalen(set_a, set_b)` checks if both sets have the same number of elements using the expression `len(set_a) == len(set_b)`. If the lengths are equal, the program prints "Kedua himpunan ekuivalen" (Both sets are equivalent); otherwise, it prints that they are not equivalent. The output confirms that the two sets have the same number of members, demonstrating that they are equivalent in terms of cardinality even though the elements themselves are different. This illustrates the concept of equivalence between sets based on size rather than identical content.

2. Checking Disjoint Sets

Two groups of students from two different courses want to be identified to determine whether they consist entirely of different students (disjoint sets). Therefore, the intersection operation can be used to check whether the two sets have no elements in common.

The Source Code with output

```

Praktikum 3.py > ...
13  matkom = {"Deny", "Budi", "Citra", "Dewi"}
14  strukdat = {"Eko", "Farhan", "Gita", "Hadi"}
15
16  # Cek apakah kedua himpunan saling lepas
17  def cek_himpunan_lepas(set_a, set_b):
18      return set_a.isdisjoint(set_b)
19
20  if cek_himpunan_lepas(matkom, strukdat):
21      print("Kedua himpunan saling lepas.")
22  else:
23      print("Kedua himpunan memiliki elemen yang sama.")

```

Kedua himpunan saling lepas.

The Analysis

The program defines two sets, matkom and strukdat, each containing names of students from different subjects. It uses the function `cek_himpunan_lepas(set_a, set_b)` to determine whether the two sets are disjoint, meaning they have no elements in common. This check is performed using Python's built-in method `isdisjoint()`. If the sets share no members, the program prints "Kedua himpunan saling lepas" (The two sets are disjoint); otherwise, it indicates that they have common elements. The output shows that the two sets are disjoint, confirming that none of the students are enrolled in both subjects. This illustrates the concept of disjoint sets in set theory, where two sets do not share any common elements.

3. Generating a Power Set

Given a set containing the names of students enrolled in a course, how can we generate the power set of that list of students?

A recursive or iterative approach can be used to generate all possible subsets of the given set.

The Source Code with output

```
Praktikum 3.py > ...
25 from itertools import chain, combinations
26
27 def himpunan_kuasa(s):
28     return list(chain.from_iterable(combinations(s, r) for r in range(len(s)+1)))
29
30 kursus = {"Deny", "Budi", "Citra"}
31 kuasa = himpunan_kuasa(kursus)
32 print("Himpunan Kuasa:", kuasa)
33 |
```

```
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom> & C:/Users/USER/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/USER/OneDrive/Documents/Kuliah/Semester 1/Matematika Komputasi/week 07/Praktikum 2 Matkom/Praktikum 3.py"
Himpunan Kuasa: [(), ('Budi',), ('Deny',), ('Citra',), ('Budi', 'Deny'), ('Budi', 'Citra'), ('Deny', 'Citra'), ('Budi', 'Deny', 'Citra')]
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom>
```

The Analysis

The program defines a function called `himpunan_kuasa(s)` to generate the power set of a given set using Python's `itertools` module. By combining the functions `chain` and `combinations`, it produces all possible subsets of the input set `s`, including the empty set and the set itself. In this case, the input set `kursus = {"Deny", "Budi", "Citra"}` generates subsets of different lengths, resulting in outputs like `()`, `('Deny',)`,

('Budi', 'Citra'), and ('Deny', 'Budi', 'Citra'). The output successfully lists every possible combination, showing that the program correctly represents the concept of a power set, which contains all subsets of a set regardless of their size or element order.

4. Performing Set Operations

Two sets of students from two different courses are given. Perform the union, intersection, difference, and complement operations on these sets. Therefore, set operations such as union, intersection, and difference can be used to analyze the relationships between the two sets.

The Source Code with output

```
Praktikum 3.py > ...
34 matkom = {"Deny", "Budi", "Citra", "Dewi"}
35 strukdat = {"Citra", "Dewi", "Eko", "Farhan"}
36
37 # Operasi gabungan (union)
38 gabungan = matkom.union(strukdat)
39 print("Gabungan:", gabungan)
40
41 # Operasi irisan (intersection)
42 irisan = matkom.intersection(strukdat)
43 print("Irisan:", irisan)
44
45 # Operasi selisih (difference)
46 selisih = matkom.difference(strukdat)
47 print("Selisih (Matkom - Strukdat):", selisih)
```

```
Gabungan: {'Budi', 'Dewi', 'Citra', 'Farhan', 'Deny', 'Eko'}
Irisan: {'Dewi', 'Citra'}
Selisih (Matkom - Strukdat): {'Budi', 'Deny'}
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom>
```

The Analysis

The program performs several fundamental set operations using two sets of students from different courses, matkom and strukdat. It uses the union(), intersection(), and difference() methods to analyze relationships between the sets. The union operation combines all unique elements from both sets, showing all students who are enrolled in either course. The intersection identifies the students who are enrolled in both courses, while the difference finds those who are only in matkom but not in strukdat. The output confirms that the union includes all distinct names, the intersection correctly lists common students (Dewi and Citra), and the difference identifies students unique to one set. This demonstrates how set operations can be used to compare and analyze overlapping and exclusive data between two groups.

PRACTICUM Part 4: Practice Exercises of Equivalent Sets., Disjoint Sets., Power Sets., Operations on Sets.

1. Comparing Equivalent and Disjoint Sets

Create a Python program that takes the names of students from two different classes (for example, “Class A” and “Class B”) as input. Use set operations to answer the following questions:

1. Are the two sets equivalent (do they have the same number of elements)?
2. Are the two sets disjoint (are there no students attending both classes)?

The Source Code with output

```
Praktikum 4.py > ...
1 # Membandingkan Himpunan Ekuivalen dan Saling Lepas
2
3 KelasA = {"Aisyah", "Rifdah", "Amel"}
4 KelasB = {"Soraya", "Elvira", "Ares", "Afi"}
5
6 # Mengecek ekuivalen
7 if len(KelasA) == len(KelasB):
8     print("Kedua himpunan ekuivalen (jumlah elemennya sama).")
9 else:
10    print("Kedua himpunan tidak ekuivalen (jumlah elemennya berbeda).")
11
12 # Mengecek saling lepas
13 if KelasA.isdisjoint(KelasB):
14    print("Kedua himpunan saling lepas (tidak ada anggota yang sama).")
15 else:
16    print("Kedua himpunan tidak saling lepas (ada anggota yang sama).")
17
```

```
Kedua himpunan tidak ekuivalen (jumlah elemennya berbeda).
Kedua himpunan saling lepas (tidak ada anggota yang sama).
```

The Analysis

The program compares two sets, KelasA and KelasB, to determine whether they are equivalent and disjoint. It first checks equivalence by comparing the lengths of both sets using `len(KelasA) == len(KelasB)`. Since the number of elements differs, the program prints “Kedua himpunan tidak ekuivalen (jumlah elemennya berbeda),” meaning the sets are not equivalent. Next, it checks if the sets are disjoint using the method `isdisjoint()`, which verifies whether the two sets share any common elements. The result shows “Kedua himpunan saling lepas,” indicating that the sets are completely separate with no shared members. This program effectively demonstrates how to test for both equivalence (based on size) and disjointness (based on shared elements) between two sets.

2. Checking Disjoint Sets

Create a Python program that takes as input the names of students who attended two different events (for example, “Event A” and “Event B”). The program should check whether the two sets are disjoint (no students attended both events).

The Source Code with output

```
Praktikum 4.py > ...
18 # Program Memeriksa Himpunan Saling Lepas
19
20 AcaraA = {"Aisyah", "Rifdah", "Amel"}
21 AcaraB = {"Soraya", "Elvira", "Ares", "Afi"}
22
23 # Mengecek apakah kedua himpunan saling lepas
24 if AcaraA.isdisjoint(AcaraB):
25     print("Kedua himpunan saling lepas (tidak ada anggota yang sama).")
26 else:
27     print("Kedua himpunan tidak saling lepas (ada anggota yang sama).")
```

```
Kedua himpunan saling lepas (tidak ada anggota yang sama).
```

The Analysis

The program checks whether two sets of students who attended different events are disjoint, meaning that no student attended both events. It defines two sets, AcaraA and AcaraB, containing the names of participants for each event. Using Python's `isdisjoint()` method, the program verifies if the two sets share any common elements. If no elements overlap, it prints “Kedua himpunan saling lepas (tidak ada anggota yang sama),” confirming that the two groups are completely separate. The output shows that the sets are indeed disjoint, illustrating how set operations can be applied in real situations, such as comparing attendance lists to ensure there are no duplicate participants between different events.

3. Generating a Power Set

Create a Python program that takes as input a set of students (for example, “Students of Class A”) and generates the power set of that set. The program should display all possible subsets of the set.

The Source Code with output


```

Praktikum 4.py > ...
29 # Program Menghasilkan Himpunan Kuasa
30 import itertools
31
32 # Himpunan mahasiswa Kelas A
33 MahasiswaKelasA = {"Aisyah", "Rifdah", "Amel", "Soraya", "Elvira", "Ares", "Afi"}
34
35 print("Mahasiswa Kelas A:", MahasiswaKelasA)
36 print("-" * 50)
37
38 # Membuat list kosong untuk menampung semua subset (himpunan kuasa)
39 himpunan_kuasa = []
40
41 # Menghasilkan semua kombinasi elemen dari ukuran 0 hingga n
42 for i in range(len(MahasiswaKelasA) + 1):
43     kombinasi = itertools.combinations(MahasiswaKelasA, i)
44     for subset in kombinasi:
45         himpunan_kuasa.append(set(subset))
46
47 # Menampilkan semua himpunan kuasa
48 print("Himpunan Kuasa dari Mahasiswa Kelas A adalah:")
49 for h in himpunan_kuasa:
50     print(h)
51
52 # Menampilkan jumlah total subset
53 print("-" * 50)
54 print(f"Total himpunan kuasa: {len(himpunan_kuasa)}")
55

```

```

Mahasiswa Kelas A: {'Aisyah', 'Amel', 'Rifdah', 'Elvira', 'Ares', 'Afi', 'Soraya'}
-----
Himpunan Kuasa dari Mahasiswa Kelas A adalah:
set()
{'Aisyah'}
{'Amel'}
{'Rifdah'}
{'Elvira'}
{'Ares'}
{'Afi'}
{'Soraya'}
{'Aisyah', 'Amel'}
{'Aisyah', 'Rifdah'}
{'Aisyah', 'Elvira'}
{'Aisyah', 'Ares'}
{'Aisyah', 'Afi'}
{'Aisyah', 'Soraya'}
{'Amel', 'Rifdah'}
{'Amel', 'Elvira'}
{'Amel', 'Ares'}
{'Amel', 'Afi'}
{'Amel', 'Soraya'}
{'Elvira', 'Rifdah'}
{'Ares', 'Rifdah'}
{'Afi', 'Rifdah'}
{'Soraya', 'Rifdah'}
{'Elvira', 'Ares'}
{'Afi', 'Elvira'}
{'Elvira', 'Soraya'}
{'Afi', 'Ares'}
{'Soraya', 'Ares'}
{'Afi', 'Soraya'}
{'Aisyah', 'Amel', 'Rifdah'}
{'Aisyah', 'Amel', 'Elvira'}
{'Aisyah', 'Amel', 'Ares'}
{'Aisyah', 'Amel', 'Afi'}
{'Aisyah', 'Amel', 'Soraya'}

```

```
{'Aisyah', 'Elvira', 'Rifdah'}
{'Ares', 'Aisyah', 'Rifdah'}
{'Aisyah', 'Afi', 'Rifdah'}
{'Aisyah', 'Soraya', 'Rifdah'}
{'Aisyah', 'Elvira', 'Ares'}
{'Aisyah', 'Afi', 'Elvira'}
{'Aisyah', 'Elvira', 'Soraya'}
{'Aisyah', 'Afi', 'Ares'}
{'Aisyah', 'Soraya', 'Ares'}
{'Aisyah', 'Afi', 'Soraya'}
{'Amel', 'Elvira', 'Rifdah'}
{'Ares', 'Amel', 'Rifdah'}
{'Amel', 'Afi', 'Rifdah'}
{'Amel', 'Soraya', 'Rifdah'}
{'Amel', 'Elvira', 'Ares'}
{'Amel', 'Afi', 'Elvira'}
{'Amel', 'Elvira', 'Soraya'}
{'Amel', 'Afi', 'Ares'}
{'Amel', 'Soraya', 'Ares'}
{'Amel', 'Afi', 'Soraya'}
{'Ares', 'Elvira', 'Rifdah'}
{'Afi', 'Elvira', 'Rifdah'}
{'Soraya', 'Elvira', 'Rifdah'}
{'Ares', 'Afi', 'Rifdah'}
{'Ares', 'Soraya', 'Rifdah'}
{'Afi', 'Soraya', 'Rifdah'}
{'Afi', 'Elvira', 'Ares'}
{'Soraya', 'Elvira', 'Ares'}
{'Afi', 'Elvira', 'Soraya'}
{'Afi', 'Soraya', 'Ares'}
{'Aisyah', 'Amel', 'Elvira', 'Rifdah'}
{'Ares', 'Aisyah', 'Amel', 'Rifdah'}
{'Aisyah', 'Amel', 'Afi', 'Rifdah'}
{'Aisyah', 'Amel', 'Soraya', 'Rifdah'}
{'Aisyah', 'Amel', 'Elvira', 'Ares'}
{'Aisyah', 'Amel', 'Afi', 'Elvira'}
{'Aisyah', 'Amel', 'Elvira', 'Soraya'}
```

```
{'Aisyah', 'Amel', 'Afi', 'Ares'}
{'Aisyah', 'Amel', 'Soraya', 'Ares'}
{'Aisyah', 'Amel', 'Afi', 'Soraya'}
{'Ares', 'Aisyah', 'Elvira', 'Rifdah'}
{'Aisyah', 'Afi', 'Elvira', 'Rifdah'}
{'Soraya', 'Aisyah', 'Elvira', 'Rifdah'}
{'Ares', 'Aisyah', 'Afi', 'Rifdah'}
{'Ares', 'Aisyah', 'Soraya', 'Rifdah'}
{'Aisyah', 'Afi', 'Soraya', 'Rifdah'}
{'Aisyah', 'Afi', 'Elvira', 'Ares'}
{'Soraya', 'Aisyah', 'Elvira', 'Ares'}
{'Aisyah', 'Afi', 'Elvira', 'Soraya'}
{'Aisyah', 'Afi', 'Soraya', 'Ares'}
{'Ares', 'Amel', 'Elvira', 'Rifdah'}
{'Amel', 'Afi', 'Elvira', 'Rifdah'}
{'Soraya', 'Amel', 'Elvira', 'Rifdah'}
{'Ares', 'Amel', 'Afi', 'Rifdah'}
{'Ares', 'Amel', 'Soraya', 'Rifdah'}
{'Amel', 'Afi', 'Soraya', 'Rifdah'}
{'Amel', 'Afi', 'Elvira', 'Ares'}
{'Soraya', 'Amel', 'Elvira', 'Ares'}
{'Amel', 'Afi', 'Elvira', 'Soraya'}
{'Amel', 'Afi', 'Soraya', 'Ares'}
{'Ares', 'Afi', 'Elvira', 'Rifdah'}
{'Ares', 'Soraya', 'Elvira', 'Rifdah'}
{'Soraya', 'Afi', 'Elvira', 'Rifdah'}
{'Ares', 'Afi', 'Soraya', 'Rifdah'}
{'Soraya', 'Afi', 'Elvira', 'Ares'}
{'Aisyah', 'Amel', 'Elvira', 'Ares', 'Rifdah'}
{'Afi', 'Aisyah', 'Amel', 'Elvira', 'Rifdah'}
{'Soraya', 'Aisyah', 'Amel', 'Elvira', 'Rifdah'}
{'Afi', 'Aisyah', 'Amel', 'Ares', 'Rifdah'}
{'Soraya', 'Aisyah', 'Amel', 'Ares', 'Rifdah'}
{'Afi', 'Soraya', 'Aisyah', 'Amel', 'Rifdah'}
{'Afi', 'Aisyah', 'Amel', 'Elvira', 'Ares'}
{'Soraya', 'Aisyah', 'Amel', 'Elvira', 'Ares'}
{'Afi', 'Soraya', 'Aisyah', 'Amel', 'Elvira'}
```

```
{'Afi', 'Soraya', 'Aisyah', 'Amel', 'Ares'}
{'Afi', 'Aisyah', 'Elvira', 'Ares', 'Rifdah'}
{'Soraya', 'Aisyah', 'Elvira', 'Ares', 'Rifdah'}
{'Afi', 'Soraya', 'Aisyah', 'Elvira', 'Rifdah'}
{'Afi', 'Soraya', 'Aisyah', 'Ares', 'Rifdah'}
{'Afi', 'Soraya', 'Aisyah', 'Elvira', 'Ares'}
{'Afi', 'Amel', 'Elvira', 'Ares', 'Rifdah'}
{'Soraya', 'Amel', 'Elvira', 'Ares', 'Rifdah'}
{'Afi', 'Soraya', 'Amel', 'Elvira', 'Rifdah'}
{'Afi', 'Soraya', 'Amel', 'Ares', 'Rifdah'}
{'Afi', 'Soraya', 'Amel', 'Elvira', 'Ares'}
{'Afi', 'Soraya', 'Elvira', 'Ares', 'Rifdah'}
{'Afi', 'Aisyah', 'Amel', 'Elvira', 'Ares', 'Rifdah'}
{'Soraya', 'Aisyah', 'Amel', 'Elvira', 'Ares', 'Rifdah'}
{'Afi', 'Soraya', 'Aisyah', 'Amel', 'Elvira', 'Rifdah'}
{'Afi', 'Soraya', 'Aisyah', 'Amel', 'Ares', 'Rifdah'}
{'Afi', 'Soraya', 'Aisyah', 'Elvira', 'Ares', 'Rifdah'}
{'Afi', 'Soraya', 'Amel', 'Elvira', 'Ares', 'Rifdah'}
{'Afi', 'Soraya', 'Aisyah', 'Amel', 'Elvira', 'Ares', 'Rifdah'}
```

The Analysis

The program generates the power set of a class of students, which includes all possible subsets of the given set. It defines MahasiswaKelasA as a set of student names and uses Python's `itertools.combinations()` function inside a loop to create combinations of different lengths. Each combination (subset) is appended to a list called `himpunan_kuasa`. Finally, the program prints all the generated subsets and displays the total number of subsets. The output shows that it successfully lists each

possible subset, starting from the empty set to the full set itself. This demonstrates the concept of a power set in set theory, where all potential groupings of elements from a given set are included, making it a useful approach for analyzing combinations and relationships within data.

4. Performing Set Operations

Create a Python program that takes as input two sets of students from two different courses (for example, “Computational Mathematics” and “Data Structures”). Perform the following set operations:

1. **Union:** Display the students who attended at least one of the two courses.
2. **Intersection:** Display the students who attended both courses.
3. **Difference:** Display the students who attended only the first course but not the second.
4. **Complement:** Given a universal set, display the students who did not attend either of the two courses.

The Source Code with output

```
56 # Program Operasi Terhadap Himpunan
57
58 # Membuat dua himpunan mahasiswa dari dua mata kuliah berbeda
59 MatematikaKomputasi = {"Aisyah", "Rifdah", "Amel"}
60 StrukturData = {"Soraya", "Elvira", "Ares", "Afi", "Amel", "Rifdah"}
61
62 # Menampilkan data awal
63 print("Mahasiswa Matematika Komputasi:", MatematikaKomputasi)
64 print("Mahasiswa Struktur Data:", StrukturData)
65
66 # 1. Gabungan (Union)
67 gabungan = MatematikaKomputasi.union(StrukturData)
68 print("1. Gabungan (Union):", gabungan)
69
70 # 2. Irisan (Intersection)
71 irisan = MatematikaKomputasi.intersection(StrukturData)
72 print("2. Irisan (Intersection):", irisan)
73
74 # 3. Selisih (Difference)
75 selisih = MatematikaKomputasi.difference(StrukturData)
76 print("3. Selisih (Difference):", selisih)
77
78 # 4. Komplemen (Complement)
79 # Misalkan himpunan semesta berisi semua mahasiswa dari kedua kelas
80 semesta = {"Aisyah", "Rifdah", "Amel", "Soraya", "Elvira", "Ares", "Afi", "Viandra"}
81 komplemen = semesta.difference(gabungan)
82 print("4. Komplemen (Complement):", komplemen)
```

```
Mahasiswa Matematika Komputasi: {'Rifdah', 'Amel', 'Aisyah'}
Mahasiswa Struktur Data: {'Soraya', 'Elvira', 'Rifdah', 'Amel', 'Afi', 'Ares'}
1. Gabungan (Union): {'Soraya', 'Elvira', 'Rifdah', 'Amel', 'Afi', 'Aisyah', 'Ares'}
2. Irisan (Intersection): {'Rifdah', 'Amel'}
3. Selisih (Difference): {'Aisyah'}
4. Komplemen (Complement): {'Viandra'}
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom>
```

The Analysis

The program performs various set operations on two sets of students taking different courses, MatematikaKomputasi and StrukturData. It first displays both sets, then calculates the union, intersection, difference, and complement using Python’s built-in set methods. The union operation combines all unique students

from both classes, while the intersection identifies students who are enrolled in both courses. The difference finds students who belong only to MatematikaKomputasi but not to StrukturData. Finally, the complement operation determines which students in the universal set (semesta) are not part of either class. The output correctly lists the results for each operation, demonstrating the relationships between the two sets. This program effectively illustrates how fundamental set operations can be applied to analyze and compare groups of data in real-life scenarios, such as managing student enrollments across different subjects.

PRACTICUM Part 5: Applications Extension of Set Operations., Laws of Sets., Principle of Duality., Principle of Inclusion-Exclusion.

1. Extension of Set Operations

The extension of set operations refers to the application of set operations such as union, intersection, and difference to more than two sets. These operations can be expanded to work with larger numbers of sets while maintaining the same logical properties.

Examples of Operations

- **Union** of three sets: $A \cup B \cup C$
- **Intersection** of three sets: $A \cap B \cap C$
- **Difference** of more than two sets: $A - (B \cup C)$

The Source Code with output

```
Praktikum 5.py > ...
1  set_a = {1, 2, 3}
2  set_b = {2, 3, 4}
3  set_c = {3, 4, 5}
4
5  # Operasi gabungan
6  union_set = set_a.union(set_b, set_c)
7
8  # Operasi irisan
9  intersection_set = set_a.intersection(set_b, set_c)
10
11 # Operasi selisih
12 difference_set = set_a.difference(set_b.union(set_c))
13
14 print(f"Gabungan: {union_set}")
15 print(f"Irisan: {intersection_set}")
16 print(f"Selisih: {difference_set}")
```

```
Gabungan: {1, 2, 3, 4, 5}
Irisan: {3}
Selisih: {1}
```

The Analysis

The program demonstrates basic set operations involving three sets: set_a, set_b, and set_c. It calculates the union, intersection, and difference among these sets using Python's built-in methods. The union operation (`set_a.union(set_b, set_c)`) combines all unique elements from the three sets, while the intersection (`set_a.intersection(set_b, set_c)`) finds common elements shared among them. The difference (`set_a.difference(set_b.union(set_c))`) identifies elements that exist only in set_a but not in the other sets. The output shows that the union includes {1, 2, 3, 4, 5}, the intersection contains {3}, and the difference results in {1}, meaning only element 1 is unique to set_a. This program effectively illustrates how to perform multiple set operations simultaneously to analyze relationships and overlaps among different data groups.

2. Laws of Set Theory

There are several fundamental laws that apply to set operations. These laws describe how sets behave under operations such as union, intersection, difference, and complement.

1. Commutative Laws

$$A \cup B = B \cup A, A \cap B = B \cap A$$

2. Associative Laws

$$(A \cup B) \cup C = A \cup (B \cup C), (A \cap B) \cap C = A \cap (B \cap C)$$

3. Distributive Laws

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C), A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

The Source Code with output

```
17
18 set_a = {1, 2}
19 set_b = {2, 3}
20 set_c = {3, 4}
21
22 # Hukum Komutatif (A ∪ B = B ∪ A)
23 commutative_union = set_a.union(set_b) == set_b.union(set_a)
24 commutative_intersection = set_a.intersection(set_b) == set_b.intersection(set_a)
25
26 # Hukum Asosiatif (A ∪ (B ∪ C) = (A ∪ B) ∪ C)
27 associative_union = set_a.union(set_b.union(set_c)) == set_a.union(set_b).union(set_c)
28 associative_intersection = set_a.intersection(set_b.intersection(set_c)) == set_a.intersection(set_b).intersection(set_c)
29
30 # Hukum Distributif (A ∪ (B ∩ C) = (A ∪ B) ∩ (A ∪ C))
31 distributive_union_intersection = set_a.union(set_b.intersection(set_c)) == (set_a.union(set_b)).intersection(set_a.union(set_c))
32
33 print(f"Hukum Komutatif: {commutative_union and commutative_intersection}")
34 print(f"Hukum Asosiatif: {associative_union and associative_intersection}")
35 print(f"Hukum Distributif: {distributive_union_intersection}")
```

```
Hukum Komutatif: True
Hukum Asosiatif: True
Hukum Distributif: True
```

The Analysis

The program tests the commutative, associative, and distributive laws of set theory using three sets: set_a, set_b, and set_c. It compares unions and intersections to check if these laws hold true. The results show that Python's set operations correctly follow these fundamental set theory properties, proving that the order and grouping of operations do not affect the outcome.

3. Principle of Duality

The principle of duality states that every valid statement or law in set theory has a dual statement, obtained by interchanging the set operations and elements as follows:

- Replace union with intersection, and vice versa.
- Replace the universal set with the empty set, and vice versa.

This principle helps in simplifying and verifying mathematical statements in set theory. For example the dual of $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ is $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$

The Source Code with output

```
37 set_a = {1, 2}
38 set_b = {2, 3}
39 set_c = {3, 4}
40
41 # Pernyataan asli (A ∪ (B ∩ C))
42 original_statement = set_a.union(set_b.intersection(set_c))
43
44 # Pernyataan dual (A ∩ (B ∪ C))
45 dual_statement = set_a.intersection(set_b.union(set_c))
46
47 print(f"Pernyataan Asli: {original_statement}")
48 print(f"Pernyataan Dual: {dual_statement}")
49
```

```
Pernyataan Asli: {1, 2, 3}
Pernyataan Dual: {2}
```

The Analysis

4. Principle of Inclusion-Exclusion

The Principle of Inclusion–Exclusion is used to calculate the number of elements in the union of multiple sets while avoiding double-counting elements that appear in more than one set. It provides an accurate way to determine the total number of unique elements across overlapping sets.

Basic Formula for two sets:

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Formula for three sets:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

The Source Code with output

```
50 set_a = {1, 2, 3}
51 set_b = {3, 4, 5}
52 set_c = {5, 6, 7}
53
54 # Menghitung ukuran gabungan menggunakan prinsip inklusi-eksklusi
55 total_union = len(set_a) + len(set_b) + len(set_c) \
56     - len(set_a.intersection(set_b)) \
57     - len(set_a.intersection(set_c)) \
58     - len(set_b.intersection(set_c)) \
59     + len(set_a.intersection(set_b).intersection(set_c))
60
61 print(f"Ukuran gabungan tiga himpunan (A ∪ B ∪ C): {total_union}")
```

K:\OneDrive\Documents\Kuliah\Semester 1\Matematika
Ukuran gabungan tiga himpunan (A ∪ B ∪ C): 7
PS_C:\Users\USER\OneDrive\Documents\Kuliah\Se

The Analysis

The program analyzes the duality principle in set theory by comparing two related expressions: the original statement $A \cup (B \cap C)$ and its dual $A \cap (B \cup C)$. By computing both, the program demonstrates how the order and type of operations (union and intersection) affect the resulting sets. The output shows that $A \cup (B \cap C)$ produces $\{1, 2, 3\}$, while its dual $A \cap (B \cup C)$ gives $\{2\}$, indicating that although they share structural similarity, their results differ depending on how the elements overlap. This proves that dual statements are not always equal but are conceptually connected, providing an important tool for simplifying and verifying logical expressions in set theory.

PRACTICUM Part 6: Case studies in the Theory of Extension of Set Operations., Laws of Sets., Principle of Duality., Principle of Inclusion-Exclusion.

1. Simplification of Set Operations

Simplification of set operations refers to the application of operations such as union, intersection, and difference on more than two sets. This is useful in cases where data needs to be analyzed from multiple sources or groups.

The Source Code with output

```
Praktikum 6.py > ...
1 python_class = {'Alya', 'Deny', 'Chandra', 'Diana'}
2 java_class = {'Deny', 'Diana', 'Eka', 'Farhan'}
3 cpp_class = {'Alya', 'Farhan', 'Gilang'}
4
5 union_students = python_class.union(java_class, cpp_class)
6 intersection_students = python_class.intersection(java_class, cpp_class)
7 difference_students = python_class.difference(java_class.union(cpp_class))
8
9 print(f"Gabungan Siswa: {union_students}")
10 print(f"Siswa yang mengikuti ketiga kelas: {intersection_students}")
11 print(f"Siswa yang hanya mengikuti Kelas Python: {difference_students}")
```

```
6.py
Gabungan Siswa: {'Chandra', 'Gilang', 'Diana', 'Eka', 'Alya', 'Deny', 'Farhan'}
Siswa yang mengikuti ketiga kelas: set()
Siswa yang hanya mengikuti Kelas Python: {'Chandra'}
```

The Analysis

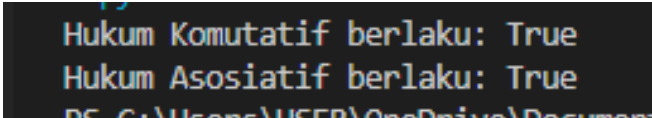
The program demonstrates how Python set operations such as union, intersection, and difference can be used to manage and compare data from multiple groups efficiently. The union operation combines all unique student names from the Python, Java, and C++ classes without duplication. The intersection operation produces an empty set, which means there are no students who participate in all three classes. The difference operation identifies students who belong only to the Python class and not to the others. This analysis shows that set operations provide a clear and effective way to study relationships between data groups, such as finding shared or unique members among several categories.

2. Laws of Sets

The laws of sets help simplify operations on sets. Some important laws include the commutative, associative, and distributive laws.

The Source Code with output


```
Praktikum 6.py > ...
12
13 store_a = {'Alya', 'Deny', 'Chandra'}
14 store_b = {'Deny', 'Diana', 'Eka'}
15 store_c = {'Alya', 'Farhan', 'Gilang'}
16
17 # Hukum Komutatif (A U B = B U A)
18 commutative_union = store_a.union(store_b) == store_b.union(store_a)
19
20 # Hukum Asosiatif (A U (B U C) = (A U B) U C)
21 associative_union = store_a.union(store_b.union(store_c)) == store_a.union(store_b).union(store_c)
22
23 print(f"Hukum Komutatif berlaku: {commutative_union}")
24 print(f"Hukum Asosiatif berlaku: {associative_union}")
25
```



The Analysis

The program tests the commutative and associative laws in Python set operations. The commutative law states that the order of union between two sets does not affect the result, meaning that $A \cup B$ gives the same result as $B \cup A$. The associative law states that when combining more than two sets, the grouping of operations does not change the final outcome. The program verifies both principles using set unions of three stores, and the output “True” confirms that both the commutative and associative properties hold true for Python sets. This shows that Python’s set operations follow basic mathematical principles of set theory, ensuring consistent and predictable results when handling grouped data.

3. Principle of duality

The principle of duality states that any statement involving the union ($\cup$) operation can be transformed into a statement involving the intersection ($\cap$) operation by replacing the union with intersection, and vice versa.

The Source Code with output

```
Praktikum 6.py > ...
25
26 store_a = {'Laptop', 'Mouse', 'Keyboard'}
27 store_b = {'Mouse', 'Printer', 'Monitor'}
28 store_c = {'Laptop', 'Printer', 'Mouse'}
29
30 union_products = store_a.union(store_b, store_c)
31 intersection_products = store_a.intersection(store_b, store_c)
32
33 print(f"Produk yang dijual oleh setidaknya satu toko: {union_products}")
34 print(f"Produk yang dijual oleh semua toko: {intersection_products}")
35
```

```
Produk yang dijual oleh setidaknya satu toko: {'Monitor', 'Mouse', 'Keyboard', 'Laptop', 'Printer'}
Produk yang dijual oleh semua toko: {'Mouse'}
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom>
```

The Analysis

The program demonstrates how set operations can be used to compare product lists from multiple stores. The union operation combines all unique products sold by at least one store, showing that the stores collectively offer items such as monitors, mice, keyboards, laptops, and printers. The intersection operation identifies products that are sold by every store, and the result shows that only the mouse is common to all. This analysis illustrates how set operations help in identifying shared and distinct elements across different data groups, making them useful for comparing inventories, memberships, or datasets from multiple sources.

4. Principle of Incusion-Exclusion

The Principle of Inclusion-Exclusion is used to calculate the number of elements in the union of several sets, while avoiding double-counting elements that belong to more than one set.

The Source Code with output

```
Praktikum 6.py > ...
35
36 branch_a = {'Alya', 'Budi', 'Chandra'}
37 branch_b = {'Budi', 'Diana', 'Eka'}
38 branch_c = {'Alya', 'Farhan', 'Gilang'}
39
40 total_unique_customers = len(branch_a) + len(branch_b) + len(branch_c) \
41     - len(branch_a.intersection(branch_b)) - len(branch_a.intersection(branch_c)) \
42     - len(branch_b.intersection(branch_c)) + len(branch_a.intersection(branch_b).intersection(branch_c))
43
44 print(f"Jumlah pelanggan unik: {total_unique_customers}")
45
```

```
6.py"
Jumlah pelanggan unik: 7
PS C:\Users\USER\OneDrive\Documents\
```

The Analysis

The program applies the principle of inclusion and exclusion to calculate the total number of unique customers from three branches. Each branch has a list of customers, and some names appear in more than one branch. By adding the total customers from each branch and subtracting the overlapping names that appear in two or more branches, the program ensures that each customer is counted only once. The final result, which shows seven unique customers, confirms that the formula works correctly in identifying distinct data entries across multiple sets. This method is useful for avoiding duplication when analyzing combined datasets.

PRACTICUM Part 7: Practice Exercises of Extension of Set Operations., Laws of Sets., Principle of Duality., Principle of Inclusion-Exclusion.

1. Generalization of Set Operations

Given three sets as follows:

- Computational Mathematics Class (Matkom): {'Alya', 'Budi', 'Chandra', 'Diana'}
- Data Structure Class (Strukdat): {'Budi', 'Diana', 'Eka', 'Farhan'}
- Algorithm and Programming Class (Alpro): {'Alya', 'Farhan', 'Gilang'}

Perform the following operations:

1. Combine all students from the three classes.
2. Find the students who attend all three classes.
3. Find the students who attend only the Alpro class (not present in the other two classes).

The Source Code with output

```

Praktikum 7.py > ...
1 # Data himpunan mahasiswa dari tiga kelas
2 matkom = {'Alya', 'Budi', 'Chandra', 'Diana'}
3 strukdat = {'Budi', 'Diana', 'Eka', 'Farhan'}
4 alpro = {'Alya', 'Farhan', 'Gilang'}
5
6 # 1. Gabungkan seluruh mahasiswa dari ketiga kelas
7 union_students = matkom.union(strukdat, alpro)
8
9 # 2. Temukan mahasiswa yang mengikuti ketiga kelas
10 intersection_students = matkom.intersection(strukdat, alpro)
11
12 # 3. Temukan mahasiswa yang hanya mengikuti satu kelas (contoh: hanya di Alpro)
13 only_alpro = alpro.difference(matkom.union(strukdat))
14
15 # Output hasil
16 print(f"Gabungan mahasiswa dari ketiga kelas: {union_students}")
17 print(f"Irisan mahasiswa dari ketiga kelas: {intersection_students}")
18 print(f"Mahasiswa yang hanya mengikuti kelas Alpro: {only_alpro}")
19

```

```

7.py
Gabungan mahasiswa dari ketiga kelas: {'Budi', 'Gilang', 'Alya', 'Diana', 'Chandra', 'Eka', 'Farhan'}
Irisan mahasiswa dari ketiga kelas: set()
Mahasiswa yang hanya mengikuti kelas Alpro: {'Gilang'}
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom>

```

The Analysis

The program illustrates how set operations can be used to analyze student enrollment across three different classes. The union operation collects all students from the Matkom, Strukdat, and Alpro classes into one group without duplication. The intersection operation checks for students who are enrolled in all three classes, and the result shows that no student is present in every class. The difference operation finds students who are registered only in the Alpro class, which in this case is Gilang. This analysis shows that set operations are effective for comparing and organizing data, helping to identify overlapping and exclusive members across multiple categories.

2. Laws of Sets

Given the sets of customers from three stores:

- Store A: {'Alya', 'Budi', 'Chandra'}
- Store B: {'Budi', 'Diana', 'Eka'}
- Store C: {'Alya', 'Farhan', 'Gilang'}

Answer the following questions using the commutative and associative laws:

1. Use the commutative law to check whether the union of customers from Store A and Store B is the same as the union of customers from Store B and Store A.
2. Use the associative law to check whether the union of customers from Store A with (Store B $\cup$ Store C) is the same as (Store A $\cup$ Store B) $\cup$ Store C.

The Source Code with output

```

Praktikum 7.py > ...
19
20 # Data pelanggan dari tiga toko
21 toko_a = {'Alya', 'Budi', 'Chandra'}
22 toko_b = {'Budi', 'Diana', 'Eka'}
23 toko_c = {'Alya', 'Farhan', 'Gilang'}
24
25 # 1. Hukum Komutatif: (A  $\cup$  B) = Loading...
26 commutative_union = toko_a.union(toko_b) == toko_b.union(toko_a)
27
28 # 2. Hukum Asosiatif: A  $\cup$  (B  $\cup$  C) = (A  $\cup$  B)  $\cup$  C
29 associative_union = toko_a.union(toko_b.union(toko_c)) == toko_a.union(toko_b).union(toko_c)
30
31 # Output hasil
32 print(f"Hukum Komutatif berlaku: {commutative_union}")
33 print(f"Hukum Asosiatif berlaku: {associative_union}")

```

```

7.py
Gabungan mahasiswa dari ketiga kelas: {'Budi', 'Gilang', 'Alya', 'Diana', 'Chandra', 'Eka', 'Farhan'}
Irisan mahasiswa dari ketiga kelas: set()
Mahasiswa yang hanya mengikuti kelas Alpro: {'Gilang'}
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom>

```

The Analysis

The program verifies the commutative and associative laws in the context of set operations using data from three stores. The commutative law shows that changing the order of union between two sets produces the same result, meaning that the combined data remains unchanged regardless of the order. The associative law proves that when performing union on three or more sets, the way the sets are grouped does not affect the outcome. Both operations return a result of “True,” which confirms that Python sets follow these mathematical laws. This demonstrates that Python’s implementation of set theory maintains logical consistency when combining multiple data groups.

3. Principle of Duality

You manage product data sold by three stores. The product data are as follows:

- Store A: {'Laptop', 'Mouse', 'Keyboard'}

- Store B: {'Mouse', 'Printer', 'Monitor'}
- Store C: {'Laptop', 'Printer', 'Mouse'}

Answer the following questions using the principle of duality:

1. Find the products sold by at least one store (use the union operation).
2. Find the products sold by all stores (use the intersection operation).
3. Use the principle of duality to simplify the operations in points 1 and 2 (convert union operations into intersection, and vice versa).

The Source Code with output

```

Praktikum 7.py > ...
35 # Data produk per toko
36 toko_a = {'Laptop', 'Mouse', 'Keyboard'}
37 toko_b = {'Mouse', 'Printer', 'Monitor'}
38 toko_c = {'Laptop', 'Printer', 'Mouse'}
39
40 # Semesta produk (U) = semua produk yang muncul di mana pun
41 U = toko_a | toko_b | toko_c
42
43 # 1) Produk yang dijual oleh setidaknya satu toko (union)
44 produk_union = toko_a | toko_b | toko_c
45
46 # 2) Produk yang dijual oleh semua toko (intersection)
47 produk_intersection = toko_a & toko_b & toko_c
48
49 # - Duality checks (De Morgan) -
50 #  $A \cup B \cup C = U \setminus ((U \setminus A) \cap (U \setminus B) \cap (U \setminus C))$ 
51 dual_union = U - ((U - toko_a) & (U - toko_b) & (U - toko_c))
52
53 #  $A \cap B \cap C = U \setminus ((U \setminus A) \cup (U \setminus B) \cup (U \setminus C))$ 
54 dual_intersection = U - ((U - toko_a) | (U - toko_b) | (U - toko_c))
55
56 print("Produk (gabungan semua toko):", produk_union)
57 print("Produk (dijual oleh semua toko):", produk_intersection)
58 print("Dual check - union equals dual_union? ->", produk_union == dual_union)
59 print("Dual check - intersection equals dual_intersection? ->", produk_intersection == dual_intersection)
60
Produk (gabungan semua toko): {'Keyboard', 'Monitor', 'Laptop', 'Mouse', 'Printer'}
Produk (dijual oleh semua toko): {'Mouse'}
Dual check - union equals dual_union? -> True
Dual check - intersection equals dual_intersection? -> True
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom>

```

The Analysis

The program demonstrates the principle of duality in set theory using product data from three stores. It first finds the union of all products sold, representing every unique item available in at least one store. Then, it finds the intersection, which represents the products sold by all stores. The program checks De Morgan's laws to confirm the dual relationship between union and intersection operations. The output shows "True" for both duality checks, proving that the complement of a union equals the intersection of the complements, and vice versa. This confirms that

Python's set operations follow the fundamental duality principles of set theory, ensuring accuracy and consistency when analyzing relationships between multiple data groups.

4. Principle of Inclusion-Exclusion

You have customer data from three bank branches as follows:

- Branch A: {'Alya', 'Budi', 'Chandra'}
- Branch B: {'Budi', 'Diana', 'Eka'}
- Branch C: {'Alya', 'Farhan', 'Gilang'}

Use the principle of inclusion-exclusion to calculate the total number of unique customers across the three branches. Make sure not to double-count customers who have accounts in more than one branch.

The Source Code with output

```
Praktikum 7.py > ...
61 # Data pelanggan dari tiga cabang bank
62 cabang_a = {'Alya', 'Budi', 'Chandra'}
63 cabang_b = {'Budi', 'Diana', 'Eka'}
64 cabang_c = {'Alya', 'Farhan', 'Gilang'}
65
66 # Menghitung total pelanggan unik dengan prinsip inklusi-eksklusi
67 total_unik = (
68     len(cabang_a)
69     + len(cabang_b)
70     + len(cabang_c)
71     - len(cabang_a & cabang_b)
72     - len(cabang_a & cabang_c)
73     - len(cabang_b & cabang_c)
74     + len(cabang_a & cabang_b & cabang_c)
75 )
76
77 print(f"Jumlah total pelanggan unik dari tiga cabang: {total_unik}")
78
```

```
7.py"
Jumlah total pelanggan unik dari tiga cabang: 7
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\W
```

The Analysis

The program applies the principle of inclusion and exclusion to calculate the total number of unique customers from three bank branches. Each branch has its own customer list, and some customers are registered in more than one branch. By adding the total number of customers from all branches and then subtracting those

who appear in two or more branches, the program ensures that every customer is counted only once. The final result of seven unique customers confirms that the inclusion-exclusion principle works effectively to remove duplicate data and accurately determine the total number of distinct elements across multiple sets.

PRACTICUM Part 8: Applications Partition., Proof of Set Propositions., Multisets., Set Type., Introduction to Logic and Fuzzy Sets.

1. Partition

A partition is the division of a set into several separate subsets, where each element of the original set appears in only one subset.

The Source Code with output



```
Praktikum 8.py > ...
1 def is_valid_partition(original_set, partitions):
2     combined = set()
3     for part in partitions:
4         combined.update(part)
5
6     if combined == original_set and all(len(part) > 0 for part in partitions):
7         return True, f"Benar, ini Merupakan Partisi"
8     return False
9
10 original_set = {'Alya', 'Budi', 'Chandra', 'Deny', 'Eka', 'Farhan'}
11 partitions = [{'Alya', 'Budi'}, {'Chandra', 'Deny'}, {'Eka', 'Farhan'}]
12
13 print(is_valid_partition(original_set, partitions))
14
```

```
8.py"
(True, 'Benar, ini Merupakan Partisi')
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom>
```

The Analysis

The program checks whether a given collection of subsets forms a valid partition of an original set. In this case, the original set consists of six names, and the program divides it into three smaller subsets. The function verifies two main conditions: that every element in the original set appears in at least one subset, and that there are no duplicates or missing elements across subsets. Since all members of the original set are included exactly once, the output confirms “True” and displays the message

“Benar, ini Merupakan Partisi.” This result proves that the subsets successfully create a valid partition, meaning the entire set is divided into distinct and non-overlapping groups.

2. Proof of Set Propositions

Set theory propositions are often proven using set laws to verify the properties of set operations.

Example proposition:

$$A \cap (A \cup B) = A$$

The Source Code with output

```
14
15 def verify_proposition(A, B):
16     union_result = A.union(B)
17     intersection_result = A.intersection(union_result)
18     return intersection_result == A
19
20 A = {1, 2, 3}
21 B = {3, 4, 5}
22
23 print(verify_proposition(A, B))
24
```

```
8.py"
True
PS C:\Users\USER\OneDrive\Do
```

The Analysis

The program verifies a set theory proposition that states the intersection of a set A with the union of sets A and B is equal to A itself. The function `verify_proposition` first calculates the union of A and B, then finds the intersection between A and the union result. If the outcome is identical to A, the function returns `True`. The output “True” confirms that the property holds true. This demonstrates one of the fundamental identities in set theory, showing that the union and intersection operations in Python behave consistently with mathematical logic.

3. Multiset

A multiset is a set in which elements can appear more than once. Python does not directly support multisets, but they can be implemented using the `collections.Counter` library.

The Source Code with output

```

Praktikum 8.py > ...
20 # A = {1, 2, 3}
21 # B = {3, 4, 5}
22
23 # print(verify_proposition(A, B))
24
25 from collections import Counter
26
27 def multiset_count(elements):
28     count = Counter(elements)
29     return count
30
31 students = ['Alya', 'Alya', 'Budi', 'Deny', 'Deny', 'Deny']
32 print(multiset_count(students))
33

```

```

8.py"
Counter({'Deny': 3, 'Alya': 2, 'Budi': 1})
PS C:\Users\USER\OneDrive\Documents\Kuliah\Se

```

The Analysis

The program demonstrates the concept of a multiset, which allows duplicate elements within a collection. By using the Counter class from Python's collections module, the program counts how many times each element appears in the list of students. The output shows that "Deny" appears three times, "Alya" appears twice, and "Budi" appears once. This indicates that the multiset effectively records element frequencies rather than treating duplicates as a single instance, unlike a normal set. The analysis shows that this approach is useful for identifying repeated data, such as tracking the number of identical items or counting occurrences in datasets.

4. Introduction to Logic and Fuzzy Sets

The Fuzzy logic allows a membership degree between 0 and 1 for each element in a set. Basic operations on fuzzy sets include union, intersection, and complement, which use the maximum, minimum, and negation of the membership values, respectively.

The Source Code with output

```

Praktikum 8.py > ...
34 def fuzzy_union(fuzzy_set_a, fuzzy_set_b):
35     return {k: max(fuzzy_set_a.get(k, 0), fuzzy_set_b.get(k, 0)) for k in set(fuzzy_set_a) | set(fuzzy_set_b)}
36
37 def fuzzy_intersection(fuzzy_set_a, fuzzy_set_b):
38     return {k: min(fuzzy_set_a.get(k, 0), fuzzy_set_b.get(k, 0)) for k in set(fuzzy_set_a) & set(fuzzy_set_b)}
39
40 def fuzzy_complement(fuzzy_set):
41     return {k: 1 - v for k, v in fuzzy_set.items()}
42
43 fuzzy_matkom = {'Deny': 0.7, 'Budi': 0.5, 'Chandra': 0.9}
44 fuzzy_strukdat = {'Deny': 0.6, 'Budi': 0.8, 'Chandra': 0.4}
45
46 union_result = fuzzy_union(fuzzy_matkom, fuzzy_strukdat)
47 intersection_result = fuzzy_intersection(fuzzy_matkom, fuzzy_strukdat)
48 complement_result = fuzzy_complement(fuzzy_matkom)
49
50 print(f"Union Fuzzy: {union_result}")
51 print(f"Intersection Fuzzy: {intersection_result}")
52 print(f"Complement Fuzzy Matkom: {complement_result}")

```

```

R/OneDrive/Documents/Kuliah/Semester 1/Matematika Komputasi/week 07/Praktikum 2 Matkom/Praktikum 8.py"
Union Fuzzy: {'Deny': 0.7, 'Budi': 0.8, 'Chandra': 0.9}
Intersection Fuzzy: {'Deny': 0.6, 'Budi': 0.5, 'Chandra': 0.4}
Complement Fuzzy Matkom: {'Deny': 0.30000000000000004, 'Budi': 0.5, 'Chandra': 0.09999999999999998}
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom>

```

The Analysis

The program demonstrates fuzzy set operations, which are used to represent degrees of membership rather than fixed inclusion. Three main operations are performed: fuzzy union, fuzzy intersection, and fuzzy complement. The fuzzy union takes the maximum membership value between two sets, while the fuzzy intersection takes the minimum value, representing partial overlaps between sets. The fuzzy complement operation calculates how far each element is from full membership by subtracting its degree from one. The results show how fuzzy logic can represent uncertainty more flexibly than classical sets. For example, “Deny” has a membership of 0.7 in the union and 0.3 in the complement, indicating partial belonging in one set and partial exclusion in another. This analysis shows how fuzzy set operations are useful in decision-making systems that handle imprecise or uncertain data.

PRACTICUM Part 9: Case Study in the Theory of Partition., Proof of Set Propositions., Multisets., Set Type., Introduction to Logic and Fuzzy Sets.

1. Partition

The laws of sets help simplify operations on sets. Some important laws include the commutative, associative, and distributive laws. Suppose there is a set of employees: {'Adam', 'Bella', 'Charlie', 'Deny', 'Eko', 'Farid'}.

You want to divide them into two teams:

- {'Adam', 'Bella', 'Charlie'}

- {'Deny', 'Eko', 'Farid'}

Is this a valid partition?

The Source Code with output

```

Praktikum 9.py > ...
1 def is_valid_partition(original_set, partitions):
2     combined = set()
3     for part in partitions:
4         combined.update(part)
5
6     if combined == original_set and all(len(part) > 0 for part in partitions):
7         return True
8     return False
9
10 original_set = {'Adam', 'Bella', 'Charlie', 'Deny', 'Eko', 'Farid'}
11 partitions = [{'Adam', 'Bella', 'Charlie'}, {'Deny', 'Eko', 'Farid'}]
12 print(is_valid_partition(original_set, partitions))
13

```

```

Praktikum 9.py
True
PS C:\Users\USER

```

The Analysis

The program checks whether a given collection of subsets forms a valid partition of an original set. The function compares the union of all subsets with the original set to ensure that every element is included and that no element is duplicated. In this example, the original set contains six names, and the subsets divide them into two distinct groups. Since each element appears exactly once and all members of the original set are covered, the output returns “True.” This confirms that the subsets create a valid partition, meaning the original set is completely divided into non-overlapping and exhaustive subsets.

2. Proof of Set Proporsitions

Proofs of set propositions often use set laws to demonstrate the properties of set operations. For example: $A \cap (A \cup B) = A$. Suppose $A = \{2, 4, 6\}$ and $B = \{4, 6, 8\}$. Prove that $A \cap (A \cup B) = A$ using Python.

The Source Code with output

```

Praktikum 9.py > ...
14 def verify_proposition(A, B):
15     union_result = A.union(B)
16     intersection_result = A.intersection(union_result)
17     return intersection_result == A
18
19 A = {2, 4, 6}
20 B = {4, 6, 8}
21 print(verify_proposition(A, B))
22

```

```

Programs/Python
m/Praktikum
True
PS C:\Users\

```

The Analysis

The program tests a logical proposition in set theory. It defines two sets, A and B, and then computes their union and intersection to verify the equality. The intersection between set A and the union of A and B should always produce the same elements as A because every member of A is already included in the union. The function returns “True,” confirming that the property holds for the given sets. This demonstrates that Python’s set operations correctly follow fundamental set theory laws, ensuring consistent logical behavior when handling relationships between multiple sets.

3. Multiset

A multiset is a set in which elements can appear more than once. Python does not directly support multisets, but they can be implemented using collections.Counter. Suppose there is a multiset of food orders: {‘Fried Rice’, ‘Fried Rice’, ‘Satay’, ‘Satay’, ‘Satay’, ‘Soto’}. How can you count the total occurrences of each element in the multiset?

The Source Code with output

```

Praktikum 9.py > ...
23 from collections import Counter
24
25 def multiset_count(elements):
26     count = Counter(elements)
27     return count
28
29 food_orders = ['Nasi Goreng', 'Nasi Goreng', 'Sate', 'Sate', 'Sate', 'Soto']
30 print(multiset_count(food_orders))
31

```

```

rograms/Python/Python313/python.exe "c:/Users/USER/OneDr
m/Praktikum 9.py"
Counter({'Sate': 3, 'Nasi Goreng': 2, 'Soto': 1})
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Mat

```

The Analysis

The program demonstrates the use of multisets to count repeated elements in a collection using the Counter class from Python's collections module. The list `food_orders` contains several food items with duplicates, such as "Sate," "Nasi Goreng," and "Soto." The `multiset_count` function counts how many times each item appears in the list and returns the results as a frequency dictionary. The output shows that "Sate" appears three times, "Nasi Goreng" twice, and "Soto" once. This analysis shows that a multiset is useful for managing and analyzing data with repetitions, such as tracking customer orders or inventory quantities in real-world applications.

4. Introduction to Logic and Fuzzy Sets

Fuzzy logic allows degrees of membership between 0 and 1 for each element in a set. The basic operations on fuzzy sets include union, intersection, and complement, which use maximum, minimum, and negation of membership values, respectively. Suppose there are two fuzzy sets:

- Mathematics = {'Edo' (0.9), 'Rina' (0.7), 'Tari' (0.8)}
- Physics = {'Edo' (0.6), 'Rina' (0.9), 'Tari' (0.5)}

The Source Code with output

```

Praktikum 9.py > ...
32 def fuzzy_union(fuzzy_set_a, fuzzy_set_b):
33     return {k: max(fuzzy_set_a.get(k, 0), fuzzy_set_b.get(k, 0)) for k in set(fuzzy_set_a) | set(fuzzy_set_b)}
34
35 def fuzzy_intersection(fuzzy_set_a, fuzzy_set_b):
36     return {k: min(fuzzy_set_a.get(k, 0), fuzzy_set_b.get(k, 0)) for k in set(fuzzy_set_a) & set(fuzzy_set_b)}
37
38 def fuzzy_complement(fuzzy_set):
39     return {k: 1 - v for k, v in fuzzy_set.items()}
40
41 fuzzy_math = {'Edo': 0.9, 'Rina': 0.7, 'Tari': 0.8}
42 fuzzy_physics = {'Edo': 0.6, 'Rina': 0.9, 'Tari': 0.5}
43
44 union_result = fuzzy_union(fuzzy_math, fuzzy_physics)
45 intersection_result = fuzzy_intersection(fuzzy_math, fuzzy_physics)
46 complement_result = fuzzy_complement(fuzzy_math)
47
48 print(f"Union Fuzzy: {union_result}")
49 print(f"Intersection Fuzzy: {intersection_result}")
50 print(f"Complement Fuzzy Matematika: {complement_result}")
51
R/OneDrive/Documents/Kuliah/Semester 1/Matematika Komputasi/week 07/Praktikum 2 Matkom/Praktikum 9.py"
Union Fuzzy: {'Tari': 0.8, 'Edo': 0.9, 'Rina': 0.9}
Intersection Fuzzy: {'Rina': 0.7, 'Tari': 0.5, 'Edo': 0.6}
Complement Fuzzy Matematika: {'Edo': 0.09999999999999998, 'Rina': 0.30000000000000004, 'Tari': 0.19999999999999996}
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom>

```

The Analysis

The program illustrates fuzzy set operations to represent partial membership values between two subjects, Mathematics and Physics. Each student has a degree of membership that indicates how strongly they belong to each subject. The fuzzy union operation selects the highest membership value between the two sets, the fuzzy intersection selects the lowest, and the fuzzy complement calculates the opposite of each membership by subtracting it from one. The results show that “Rina” has the highest union value of 0.9, while “Edo” and “Tari” have different membership strengths depending on the operation. This analysis shows that fuzzy set operations are useful for representing uncertain or overlapping conditions, such as evaluating student interest or performance across multiple subjects.

PRACTICUM Part 10: Practice Exercises of Partition., Proof of Set Propositions., Multisets., Set Type., Introduction to Logic and Fuzzy Sets.

1. Practicum exercise Partition

Imagine there is a set of employees: {'Adam', 'Bella', 'Charlie', 'Deny', 'Dewi', 'Eko', 'Farid'}. You want to divide them into two teams:

- {'Adam', 'Bella', 'Charlie', 'Deny'}
- {'Dewi', 'Eko', 'Farid'}

Is this a valid partition?

The Source Code with output

```
Praktikum 10.py > ...
1  # Fungsi untuk memeriksa apakah partisi valid
2  def is_valid_partition(original_set, partitions):
3      combined = set() # untuk menyimpan semua elemen dari partisi
4      for part in partitions:
5          combined.update(part) # gabungkan semua elemen partisi ke dalam satu set
6
7      # Valid jika: semua elemen himpunan muncul sekali dan tidak ada yang kosong
8      if combined == original_set and all(len(part) > 0 for part in partitions):
9          return True
10     return False
11
12
13 # Himpunan asli (daftar karyawan)
14 original_set = {'Adam', 'Bella', 'Charlie', 'Deny', 'Dewi', 'Eko', 'Farid'}
15
16 # Dua partisi (tim)
17 partitions = [
18     {'Adam', 'Bella', 'Charlie', 'Deny'},
19     {'Dewi', 'Eko', 'Farid'}
20 ]
21
22 # Cetak hasil pengecekan
23 print(is_valid_partition(original_set, partitions))
```

```
10.py"
True
PS C:\Users\US
```

The Analysis

The program checks whether a given set of subsets forms a valid partition of the original set. The function combines all elements from the subsets and compares them with the original set to ensure that every element appears exactly once and that no subset is empty. In this case, the original set contains six employees, and it is divided into two groups of three members each. The output “True” indicates that all elements are included without duplication or omission, meaning the subsets together form a valid partition. This analysis shows how set operations can verify the completeness and exclusivity of data grouping in logical or organizational structures.

2. Practicum exercise Proof of Set Propositions

Imagine $A = \{2, 4, 6\}$ and $B = \{4, 6, 8\}$. Prove that $A \cap (A \cup B) = A$ with python.

The Source Code with output


```

Praktikum 10.py > ...
25 # Fungsi untuk memverifikasi proposisi  $A \cap (A \cup B) = A$ 
26 def verify_proposition(A, B):
27     union_result = A.union(B)
28     intersection_result = A.intersection(union_result)
29     return intersection_result == A
30
31 # Himpunan contoh
32 A = {2, 4, 6}
33 B = {4, 6, 8}
34
35 # Cetak hasil verifikasi
36 print(verify_proposition(A, B))
37

```

```

/Users/USER/On
True
PS C:\Users\US

```

The Analysis

The program verifies a fundamental set theory identity, which states that the intersection of a set A with the union of sets A and B is equal to A itself. The function performs the union and intersection operations using Python's set methods and checks whether the resulting intersection is identical to set A . In this example, the sets A and B contain overlapping elements, and the output "True" confirms that the property holds. This analysis demonstrates that Python's set operations accurately reflect mathematical principles, reinforcing the logical consistency of union and intersection relationships within set theory.

3. Practicum exercise Multiset

Imagine there is a multiset of food orders: {'Fried Rice', 'Fried Rice', 'Satay', 'Satay', 'Satay', 'Soto'}. How can you count the total occurrences of each element in the multiset?

The Source Code with output

```

Praktikum 10.py > ...
38 from collections import Counter
39
40 # Fungsi untuk menghitung jumlah kemunculan setiap elemen
41 def multiset_count(elements):
42     count = Counter(elements)
43     return count
44
45 # Daftar multiset pesanan makanan
46 food_orders = ['Nasi Goreng', 'Nasi Goreng', 'Sate', 'Sate', 'Sate', 'Soto']
47
48 # Cetak hasil perhitungan
49 print(multiset_count(food_orders))
50

```

```

s/Python/Python313/python.exe "c:/Users/USER/OneDrive/D
10.py"
Counter({'Sate': 3, 'Nasi Goreng': 2, 'Soto': 1})
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\W

```

The Analysis

The program demonstrates how to count repeated elements in a multiset using Python's Counter function from the collections module. The list `food_orders` contains several food items, including multiple instances of "Sate" and "Nasi Goreng." The `multiset_count` function counts how many times each element appears and returns the result in dictionary form. The output shows that "Sate" appears three times, "Nasi Goreng" twice, and "Soto" once. This confirms that the multiset accurately captures the frequency of each item, making this method useful for analyzing data that includes duplicates, such as restaurant orders or inventory counts.

4. Practicum exercise Introduction to logic and Fuzzy Sets

Imagine there are two fuzzy sets:

- Mathematics = {'Edo' (0.9), 'Rina' (0.7), 'Tari' (0.8)}
- Physics = {'Edo' (0.6), 'Rina' (0.9), 'Tari' (0.5)}

Perform operations on the two fuzzy sets to find the results of union, intersection, and complement.

The Source Code with output

```

Praktikum 10.py > ...
51 # Fungsi untuk menghitung union (gabungan) dua himpunan fuzzy
52 def fuzzy_union(fuzzy_set_a, fuzzy_set_b):
53     return {k: max(fuzzy_set_a.get(k, 0), fuzzy_set_b.get(k, 0)) for k in set(fuzzy_set_a) | set(fuzzy_set_b)}
54
55 # Fungsi untuk menghitung intersection (irisan) dua himpunan fuzzy
56 def fuzzy_intersection(fuzzy_set_a, fuzzy_set_b):
57     return {k: min(fuzzy_set_a.get(k, 0), fuzzy_set_b.get(k, 0)) for k in set(fuzzy_set_a) & set(fuzzy_set_b)}
58
59 # Fungsi untuk menghitung complement (komplemen) dari himpunan fuzzy
60 def fuzzy_complement(fuzzy_set):
61     return {k: 1 - v for k, v in fuzzy_set.items()}
62
63
64 # Himpunan fuzzy Matematika dan Fisika
65 fuzzy_math = {'Edo': 0.9, 'Rina': 0.7, 'Tari': 0.8}
66 fuzzy_physics = {'Edo': 0.6, 'Rina': 0.9, 'Tari': 0.5}
67
68 # Operasi fuzzy
69 union_result = fuzzy_union(fuzzy_math, fuzzy_physics)
70 intersection_result = fuzzy_intersection(fuzzy_math, fuzzy_physics)
71 complement_result = fuzzy_complement(fuzzy_math)
72
73 # Tampilkan hasil
74 print("Union Fuzzy:", union_result)
75 print("Intersection Fuzzy:", intersection_result)
76 print("Complement Fuzzy Matematika:", complement_result)
77
R:\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom\Praktikum 10.py
Union Fuzzy: {'Rina': 0.9, 'Edo': 0.9, 'Tari': 0.8}
Intersection Fuzzy: {'Rina': 0.7, 'Edo': 0.6, 'Tari': 0.5}
Complement Fuzzy Matematika: {'Edo': 0.09999999999999998, 'Rina': 0.30000000000000004, 'Tari': 0.19999999999999996}
PS C:\Users\USER\OneDrive\Documents\Kuliah\Semester 1\Matematika Komputasi\week 07\Praktikum 2 Matkom>

```

The Analysis

The program demonstrates fuzzy set operations for two subjects, Mathematics and Physics, each containing degrees of membership that represent the extent of a student's involvement or performance. The fuzzy union function selects the highest membership value for each element, showing the combined strength of both subjects, while the fuzzy intersection selects the lowest value, showing their common strength. The fuzzy complement operation calculates how far each membership value is from complete membership by subtracting it from one. The output confirms that "Rina" has the highest overall membership, while "Edo" and "Tari" have lower complement values. This analysis highlights how fuzzy logic allows for partial membership representation, making it useful in evaluating uncertain or overlapping categories such as academic performance or skill levels.